

Agenda

- Function and their types
- Prototype
- Class

Arrow Functions

- Arrow functions are a modern, concise way to write function expressions in JavaScript, introduced in ECMAScript 2015 (ES6).
- They provide a shorter syntax than traditional function expressions, making the code more readable, especially for simple, one-line functions.
- An arrow function has three main parts:
 1. Parameters: Enclosed in parentheses ().
 2. The "fat arrow": The `=>` syntax.
 3. Function body: The code to be executed.
- They can be one-liner or multiline. However, one-liners are preferred.

```
const add = (a, b) => {  
  const res = a + b  
  console.log(res)  
}  
  
// Omitting parentheses for a single parameter:  
const square = n => {  
  const res = n * n  
  console.log(res)  
}  
  
//Omitting curly braces and return for a single expression: For one-line arrow  
functions, the return is implicit.  
const square = x => x * x;  
  
//Returning an object literal: To implicitly return an object, you must wrap the  
curly braces in parentheses to avoid a syntax error.  
const createObject = () => ({ message: "Hello" });
```

- If the function body has multiple lines, you must use curly braces and an explicit return statement.

Closure

- A closure in JS is the combination of a function and the lexical environment within which that function was declared.
- This environment consists of any variables that were in scope at the time the function was created.
- Closures allow a function to access variables from its outer scope even after that scope has finished executing

```
// function closure
function outer() {
  let z = 1;
  // inner/nested fn -- accessing local var of outer fn
  function inner(x, y) {
    return x + y + z;
  }
  return inner;
}

// call outer fn & collect its return value
const infn = outer();
// call inner fn
const result = infn(22, 7);
console.log("final closure result = " + result);
```

```
function counter() {
  let count = 0;
  return () => {
    count++;
    return count;
  }
}
const incrementCounter = counter()

console.log(incrementCounter())
console.log(incrementCounter())
console.log(incrementCounter())
```

Currying

- Currying is an advanced functional programming technique that transforms a function with multiple arguments into a sequence of nested functions, each taking a single argument.
- Instead of calling a function like `f(a, b, c)`, a curried function is called as `f(a)(b)(c)`.
- This process is made possible in JavaScript through the use of closures, which allow the inner functions to retain access to the arguments of their outer functions' scopes.

```
// function HrsToMin(n) {
//   return n * 60
// }

// function MinToSec(n) {
//   return n * 60
// }

// function KmToM(n) {
//   return n * 1000
// }
```

```
// }

function multiply(n) {
  return (value) => {
    return value * n
  }
}

const HrsToMin = multiply(60)
const MinToSec = multiply(60)
const KmToM = multiply(1000)

console.log(HrsToMin(2))
console.log(MinToSec(20))
console.log(KmToM(5))
```

Higher Order Functions

- A higher-order function (HOF) is a function that either takes one or more functions as arguments, returns a function, or both.
- This is possible in JavaScript because functions are "first-class citizens," meaning they can be treated like any other value (such as a string or number).
- A function that is passed as an argument to a higher-order function is known as a callback function.
- This is a common pattern used to handle asynchronous operations and create reusable code.

```
//executer function
// Higher Order functions
function executer(n1, n2, fn) {
  const res = fn(n1, n2)
  console.log('res - '+res)
}

const add = (n1,n2)=>{
  const res = n1+n2;
  return res;
}
// Here add is called as callback function
executer(10,20,add)

// we can pass the callback function by directly declaring it in the argument
executer(10,20,(n1,n2)=>n1-n2)
executer(10,20,(n1,n2)=>n1*n2)
```

Function Hidden Parameters

- Every function in JS accepts 2 hidden parameters
 1. this:
 - refers the current object on which the function is called

- if function is called on an instance, the instance becomes this
- E.g.

```
var p = new Object
p.myCanVote = canVote;

// the variable p becomes this inside canVote()
p.myCanVote()
```

- if function is called without an instance then Window becomes this

```
function canVote() { /* body */}

// inside canVote Window becomes this
canVote();
```

2. arguments:

- is of type array
- array of parameter values passed to a function
- E.g.

```
function add() {
  var answer = 0;
  for (var index = 0; index < arguments.length; index++) {
    answer += arguments[index];
  }
  console.log(answer)
}
```

Difference between arrow functions and normal function

- Arrow functions are a modern, more concise alternative introduced in ES6, while normal functions have been part of JavaScript since its inception.
- this keyword behavior : This is the most significant difference between the two function types.
- Normal functions: The value of this is dynamic and depends on how the function is called.
 - As an object method: this refers to the object that owns the method.
 - As a standalone function: this defaults to the global object (window in browsers) or is undefined in strict mode.
- Arrow functions: They do not have their own this binding. Instead, they inherit the this value from the parent or enclosing lexical scope where they were defined. This behavior is predictable and avoids common issues with this losing its context in callbacks.

```
const user = {
  name: 'Alex',
  // Normal function: 'this' refers to the 'user' object
  regularMethod: function() {
    console.log(this.name); // Logs: Alex
  },
  // Arrow function: 'this' is inherited from the global scope
  arrowMethod: () => {
    console.log(this.name); // Logs: undefined
  }
};

user.regularMethod();
user.arrowMethod();
```

Hoisting

- In C/C++, variables or functions or classes must be declared before their use.
- Hoisting in JavaScript is a behavior where variable and function declarations are moved to the top of their scope before code execution.
- It allows using variables and functions before they are declared in the code. However, only declarations are hoisted, not initializations or assignments.

Variable Hoisting

- Variables declared with var are hoisted to the top of their scope and initialized with undefined.

```
console.log(x); // Output: undefined
var x = 5;
console.log(y); // Output: ReferenceError: Cannot access 'y' before
initialization
let y = 10;
```

Function Hoisting

- Function declarations are fully hoisted, allowing them to be called before they are defined in the code.

```
greet("Sunbeam"); // Output: Hello, Sunbeam!
function greet(name) {
  console.log("Hello, " + name + "!");
}
```

Class Hoisting

- Classes are also hoisted, but unlike function declarations, they are not initialized.

- Therefore, you cannot use a class before it is declared, similar to let and const.

```
const myClass = new MyClass();  
// Output: ReferenceError: Cannot access 'MyClass' before initialization  
  
class MyClass {  
  constructor() {  
    console.log("MyClass constructor");  
  }  
}
```

Object

- Everything in JS is an Object, even functions are also objects
- Object is a collection of properties (data members or fields) and methods
- Objects are dynamic, meaning you can add, modify, or delete properties after they are created.
- Properties can be accessed/added using dot operator or [] subscript.
- To create an object (instance) in JS, we can use
 1. { } (Object Literal)
 2. new Object()
 3. constructor function

1. { } Using Object Literal

- Create object using JSON syntax.

```
// create object & props in JS -- (using {})  
let i = {  
  name: "Pencil",  
  company: "Natraj",  
  price: 5.0,  
};  
  
console.log("type of i = " + typeof(i));  
console.log("i name = ", i.name);  
console.log("i company = ", i.company);  
console.log("i price = ", i["price"]);
```

2. Using new Object().

- Create object using new Object()
- Properties can be added dynamically or accessed using dot/[] operator.

```
// create object & props in JS -- (using .)  
let p = new Object();  
p.name = "James Bond";
```

```
p.age = 65;
p.addr = "London";
console.log("type of p = " + typeof(p));
console.log("p name = ", p.name);
console.log("p age = ", p.age);
console.log("p addr = ", p.addr);

// create object & props in JS -- (using [])
let m = new Object();
m["model"] = "iPhone 14 Pro";
m["company"] = "Apple";
m.price = 89000.0;
console.log("type of m = " + typeof (m));
console.log("m model = ", m["model"]);
console.log("m company = ", m.company);
console.log("m price = ", m["price"]);
```

3. Using constructor function

- Create object using constructor function i.e. new FnName(args).
- Properties are added into fn using "this" pointer (which refers to newly created object).
- To add common properties & methods for all objects use prototype

```
// using construction function
// Starting a constructor function name with a capital letter is a naming
// convention, not a technical requirement enforced by the JavaScript engine.
// Helps to Distinguish from regular functions
function Car(model, company, price) {
  this.model = model;
  this.company = company;
  this.price = price;
}

// add properties and methods for all objects using "prototype".
Car.prototype.color = "red";
Car.prototype.printInfo = function () {
  console.log("car model = ", this.model);
  console.log("car company = ", this.company);
  console.log("car price = ", this.price);
  console.log("car color = ", this.color);
};

// Create Car object
let c1 = new Car("i10", "Hundai", 800000.0);
c1.printInfo();
// Create another Car object
let c2 = new Car("Punch", "Tata", 900000.0);
c2.printInfo();
```

Prototype Inheritance

- In JavaScript, prototype inheritance is a mechanism that allows objects to inherit properties and methods from other objects.
- It is the core of JavaScript's object-oriented nature, even with the introduction of the modern class syntax in ES6, which is "syntactical sugar" over the same underlying prototype system.
- Key concepts of prototypal inheritance
 1. Prototypes: Every JavaScript object has an internal, hidden property called `[[Prototype]]`, which is a link to another object. That linked object is the first object's prototype.
 2. Prototype chain: When you try to access a property or method on an object, JavaScript first looks for it on the object itself. If it isn't found, it searches the object's prototype. This continues up the "prototype chain" until the property is found or the end of the chain is reached. The end of the chain is `Object.prototype`, which has a prototype of `null`.
 3. Inheritance is delegation: This system is sometimes described as delegation rather than traditional inheritance. A child object does not get its own copy of the parent's properties and methods. Instead, it delegates calls to the properties and methods stored on the parent object (its prototype).

Working with constructor functions

- Before the `class` keyword was introduced, prototype inheritance was commonly achieved using constructor functions.

```
// A "parent" constructor function
function Person(name) {
  this.name = name;
}

// Add a method to the prototype of Person
Person.prototype.display = function() {
  console.log(`Name - ${this.name}`);
};

// A "child" constructor function
function Employee(id, name, salary) {
  Person.call(this, name); // Call the parent constructor to set 'name'
  this.id = id;
  this.salary = salary;
}

// Set Employee's prototype to inherit from Person's prototype
// This creates the prototype chain: e1 -> Employee.prototype -> Person.prototype
// -> Object.prototype
Object.setPrototypeOf(Employee.prototype, Person.prototype);

// Create a new instance
const e1 = new Employee(1, "Anil", 10000);

// The Employee object does not have its own `display` method, but it can call the
// one from its prototype
e1.display(); // Output: Name - Anil
```

Key Features of Prototype

1. Underlying mechanism: Even when using the modern class syntax, JavaScript is fundamentally a prototype-based language.
2. Code reuse: Prototypes are a powerful way to share methods and properties among objects without needing to create separate copies for each instance.
3. Dynamic and flexible: The prototype chain can be modified at runtime, allowing for dynamic behavior. However, this feature should be used with caution, as it can lead to unexpected side effects.

class

- Introduced in ECMAScript 2015 (ES6), the class keyword is a syntactic sugar over JavaScript's existing prototype-based inheritance.
- While classes do not change the fundamental nature of JavaScript objects, they provide a much cleaner and more familiar syntax for creating object blueprints, inheriting from other classes, and using object-oriented programming (OOP) principles.
- Key features of JavaScript classes
 1. constructor() method: A special method called automatically when a new object is created using the new keyword. It is used to initialize the object's properties.
 2. new keyword: Creates a new instance of a class, triggering the constructor() method. Attempting to call a class without new will throw an error.
 3. Methods: Functions defined inside the class body, which provide behavior for objects created from the class.
 4. Inheritance: Classes support inheritance using the extends keyword. This allows a new class (the child) to inherit the properties and methods of an existing class (the parent).
- The super() method is used inside a child class's constructor to call the parent's constructor.

```
class Person {
  constructor(name, age, addr) {
    this.name = name;
    this.age = age;
    this.addr = addr;
  }
  printInfo() {
    console.log("person name = ", this.name);
    console.log("person age = ", this.age);
    console.log("person addr = ", this.addr);
  }
}

// create objects
let p1 = new Person("James Bond", 65, "London");
console.log("p1 type = ", typeof p1); // object type
p1.printInfo();

// JS class inheritance
class Employee extends Person {
  constructor(name, age, addr, sal, dept) {
    super(name, age, addr);
  }
}
```

```
    this.sal = sal;
    this.dept = dept;
  }
  showInfo() {
    super.printInfo()
    console.log("emp sal = ", this.sal);
    console.log("emp dept = ", this.dept);
  }
}
```

- Static methods: Defined with the static keyword, these methods are called on the class itself, not on an instance of the class. They are useful for utility functions.

```
class Utils {
  static logMessage(message) {
    console.log(message);
  }
}
Utils.logMessage("Hello from a static method!");
```

- Private fields: Since ECMAScript 2022, private fields can be declared using a hash (#) prefix. This makes a property or method only accessible from inside the class itself, ensuring better encapsulation.

```
class BankAccount {
  #balance; // This is a private field

  constructor(initialBalance) {
    this.#balance = initialBalance;
  }

  deposit(amount) {
    this.#balance += amount;
  }

  getBalance() {
    return this.#balance;
  }
}
```